RAG Pipeline for Real-Time Telemetry Query System

1. Introduction

Modern systems such as smart factories, digital twins, and IoT-based platforms generate continuous streams of real-time telemetry data from sensors and devices. This data is usually presented as raw logs or numerical values, which makes it difficult for operators to quickly understand system behavior.

The objective of this project is to design and implement an intelligent query system that allows users to interact with real-time telemetry data using natural language. Instead of manually inspecting logs or dashboards, users can ask questions such as “What is the status of the power switch?” or “When was the sensor last active?” and receive meaningful responses.

To achieve this, the project uses a Retrieval-Augmented Generation (RAG) pipeline integrated with a Large Language Model (LLM). In this design, the LLM plays a central role throughout the entire query-resolution process, including interpreting user intent, identifying relevant telemetry topics, retrieving contextual information, and generating the final response. Rather than storing all real-time telemetry values in the vector database, the system stores semantic descriptions of telemetry topics. When a query is issued, the system retrieves the real-time value directly from the MQTT data stream after the LLM identifies the topic of interest. This approach improves efficiency while maintaining accurate real-time responses.

2. Business Requirements

The system must be capable of collecting real-time telemetry data from sensors or simulated IoT logs. This data should be transmitted through a messaging system (such as MQTT) to ensure efficient and continuous data flow. The system must store telemetry data in a way that supports fast retrieval and querying.

Users must be able to interact with the system using natural language rather than technical commands. The system should understand user questions related to system status, historical behavior, and basic control commands. The responses provided must be accurate, clear, and based on the most relevant telemetry data available at the time of the query.

The system should ensure that raw telemetry data is not exposed directly to users. Instead, responses should be summarized or interpreted into human-readable form. This improves usability and reduces the risk of misinterpretation.

Scalability is also an important requirement. The system should be designed in a modular way so that additional sensors, telemetry sources, or data types can be added in the future without requiring major architectural changes.

Reliability is another key requirement. The system should continue functioning even if telemetry volume increases or if individual components experience minor failures. The design should allow for future optimization in terms of performance and accuracy.

3. User Stories

As a system operator, I want to ask questions about the current status of sensors so that I can quickly understand how the system is performing without analyzing raw logs.

As a system operator, I want to know when a sensor or component last changed state so that I can investigate abnormal behavior or system failures.

As a system operator, I want to issue simple control commands using natural language so that I can manage system components more easily.

As a developer, I want telemetry data to be stored in a searchable and structured format so that relevant information can be retrieved efficiently for user queries.

As a researcher, I want to evaluate how different language models perform when answering telemetry-related questions so that the most suitable model can be selected for the system.

C13 fx

	A	B	C
1	Title	Weeks	User Story
2	Telemetry Data Ingestion	Week 1	As a system developer, I want to ingest telemetry data from sensors using MQTT, so that real-time system data can be collected for processing.
3	Store Telemetry Data	Week 2	As a system developer, I want to store telemetry data in a database, so that both current and historical data are preserved.
4	Format and Chunk Telemetry Data	Week 3	As a developer, I want to convert telemetry records into formatted text chunks, so that the data can be used for embedding and retrieval.
5	Generate Embeddings	Week 4	As a developer, I want to generate vector embeddings from telemetry data, so that semantic search can be performed on it.
6	Build Vector Database	Week 5	As a system developer, I want to store embeddings in a vector database, so that relevant telemetry data can be retrieved efficiently.
7	Natural Language Query Interface	Week 6	As a system operator, I want to ask questions about telemetry using natural language, so that I do not need to analyze raw system logs.
8	Retrieve Relevant Telemetry	Week 7	As a system operator, I want the system to retrieve relevant telemetry records, so that answers are based on real system data.
9	Generate AI-Based Response	Week 8	As a system operator, I want the system to generate responses using an LLM, so that I receive clear and meaningful answers.
10	Command Execution	Week 9	As a system operator, I want to issue simple control commands using natural language, so that I can control system components easily.
11	System Integration and Testing	Week 10	As a project team, I want to integrate all system components and test them, so that the complete system works as expected.

C13 fx

	A	B	D
1	Title	Weeks	Acceptance Criteria
2	Telemetry Data Ingestion	Week 1	Given that sensors publish telemetry data, when the system subscribes to MQTT topics, then telemetry data should be received successfully.
3	Store Telemetry Data	Week 2	Given that telemetry data is received, when it is processed, then it should be stored in the system database.
4	Format and Chunk Telemetry Data	Week 3	Given that telemetry data exists, when it is processed, then it should be converted into readable text format.
5	Generate Embeddings	Week 4	Given that telemetry text chunks exist, when the embedding model is applied, then vector representations should be created.
6	Build Vector Database	Week 5	Given that embeddings exist, when they are inserted into the vector database, then they should be indexed correctly.
7	Natural Language Query Interface	Week 6	Given that the user submits a natural language query, when the query is received, then it should be processed by the system.
8	Retrieve Relevant Telemetry	Week 7	Given that a query is submitted, when retrieval is performed, then matching telemetry data should be selected from the vector database.
9	Generate AI-Based Response	Week 8	Given that telemetry data is retrieved, when it is sent to the LLM, then the LLM should generate a response using that data.
10	Command Execution	Week 9	Given that a valid command is detected, when the command is processed, then it should be sent to the MQTT broker.
11	System Integration and Testing	Week 10	Given that all modules are implemented, when integration testing is performed, then the system should work as an integrated solution.

UserStories_Mindmesh

4. System Architecture

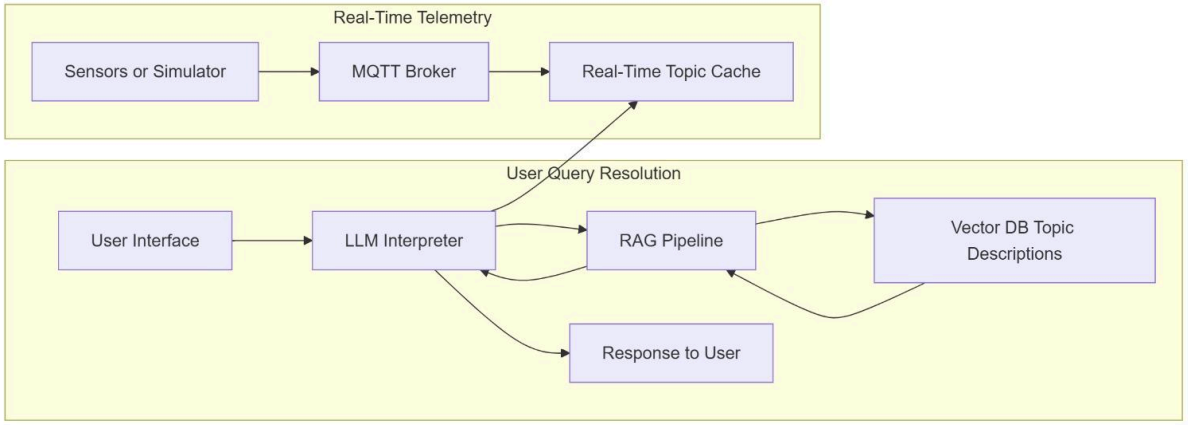
The system architecture is designed to support real-time telemetry ingestion, semantic topic retrieval, intelligent query interpretation, and natural language interaction.

Telemetry data is generated by sensors or simulated IoT devices and transmitted using an MQTT broker. The MQTT broker acts as an intermediary layer that ensures efficient and continuous data flow while preventing raw telemetry from being directly exposed to users. Incoming telemetry is cached or stored as real-time topic values.

User queries are submitted through a natural language interface and passed to the LLM, which serves as the main interpreter of the query-resolution cycle. The LLM analyzes the user query to determine the user's intent, the telemetry topic being referenced, and the type of information requested.

The RAG pipeline is used to retrieve semantic descriptions of telemetry topics from a vector database. These topic descriptions provide contextual meaning to the LLM and help it match user queries to the correct telemetry source. Once the relevant topic is identified, the system retrieves the current value of that topic directly from the real-time MQTT data stream.

The LLM then combines the retrieved topic description and real-time value to curate a human-readable response. This response is sent back to the user through the user interface. This architecture allows the system to remain modular, scalable, and efficient while supporting real-time interaction.



5. UI Design

The user interface will be designed with simplicity and clarity in mind. It will provide a text input field where users can enter natural language questions. Below the input field, a response area will display the system's answers.

The interface may also show basic system information such as query history or timestamps of responses. The primary focus of the UI is functionality rather than advanced visualization. The goal is to make interaction with telemetry data as easy as chatting with a virtual assistant.

If future extensions are required, the UI can be enhanced with dashboards or visual indicators of sensor states. However, for the current scope of the project, a lightweight text-based interface is sufficient.

6. Database Design

The system uses a vector database to store semantic descriptions of MQTT topics rather than storing raw real-time telemetry values. Each database entry represents a telemetry topic and contains attributes such as topic name, description, and metadata, along with a vector embedding generated by an embedding model.

This design allows the system to perform semantic searches on topic descriptions when processing user queries. Instead of continuously embedding high-frequency telemetry values, the system retrieves only the relevant topic description from the vector database and fetches the real-time value directly from the MQTT data stream. This reduces database update overhead and improves system performance.

The database structure supports efficient topic identification and can be easily extended when new telemetry topics are added to the system.

7. Task Division

The project tasks will be divided among team members to ensure efficient development and balanced workload.

One team member will be responsible for implementing telemetry data ingestion using MQTT and Python. This includes simulating sensor data and ensuring proper message transmission.

Another team member will focus on building the RAG pipeline and managing the vector database. This includes data chunking, embedding generation, and retrieval logic.

A third team member will integrate the LLM and handle query processing. This includes designing prompt templates and testing different LLM configurations using Ollama.

A fourth team member will be responsible for user interface development and system testing. This includes ensuring that user queries are handled correctly and responses are displayed clearly.

All team members will collaborate on documentation, evaluation, and final reporting.